

SAFETYVERSE · BY UPL

# Tech Stack & Complete File Guide

---

SafetyVerse · by UPL

---

**Version:** 2026-09 · **Audience:** All end users

**Live app:** <https://upl-fire-portal.onrender.com>

**Repository:** <https://github.com/paramshah1903/fire-portal>

---

# SafetyVerse — Tech Stack & Complete File Guide

*A beginner-friendly walkthrough of every technology used and every file in the codebase. Read it top to bottom, or skip to any section.*

## PART 1 — TECH STACK

### 1.1 The big picture

Think of SafetyVerse as three cooperating machines:

| Machine                   | What it does                                          | Analogy                         |
|---------------------------|-------------------------------------------------------|---------------------------------|
| <b>Frontend</b> (browser) | Shows the screens users interact with                 | A waiter taking your order      |
| <b>Backend</b> (server)   | Does the work — talks to database, checks permissions | The kitchen preparing food      |
| <b>Database</b>           | Remembers everything permanently                      | The restaurant's filing cabinet |

We use a different set of tools for each. Everything in the middle column below is free and open-source unless noted.

### 1.2 Frontend stack

The frontend lives in `frontend/` and runs entirely in the user's browser. It is a **Single-Page Application** (SPA) — one HTML file that loads all the app logic once, then swaps pages in and out without reloading.

| Tool                | Version | What it does                                                         | Why we chose it                                              |
|---------------------|---------|----------------------------------------------------------------------|--------------------------------------------------------------|
| <b>React</b>        | 18      | The core UI library — components, state, re-rendering                | Industry standard; huge community; easy to hire for          |
| <b>TypeScript</b>   | 5.6     | Adds types on top of JavaScript so mistakes are caught at write-time | Turns "runtime crash" into "editor squiggle"                 |
| <b>Vite</b>         | 5       | Builds and serves the frontend (dev server + production bundler)     | Faster than the older Webpack/Create-React-App               |
| <b>Tailwind CSS</b> | 3.4     | Styling via short class names ( <code>px-4 bg-white</code> etc.)     | No global CSS files to fight over; easy dark mode            |
| <b>React Router</b> | 6       | URL → page mapping in the SPA                                        | The de-facto SPA router                                      |
| <b>Axios</b>        | 1.7     | Sends HTTP requests to the backend                                   | Simpler than <code>fetch</code> for cookies + error handling |
| <b>html5-qrcode</b> | 2.3     | Turns the phone camera into a QR scanner                             | Works in any browser, no app install needed                  |
| <b>qrcode.react</b> | 4.2     | Draws QR labels for printing                                         | Renders as SVG so labels print sharp                         |
| <b>Recharts</b>     | 3       | Dashboard charts (bar / pie / line)                                  | Built specifically for React                                 |

### Development-only extras

- **ESLint** — catches common JavaScript mistakes
- **Autoprefixer + PostCSS** — makes Tailwind actually work

## 1.3 Backend stack

The backend lives in `backend/` and runs on **Node.js** (a program that executes JavaScript outside the browser). It exposes an **HTTP API** — a set of URLs the frontend calls to fetch or change data.

| Tool                      | Version | What it does                                          | Why we chose it                                         |
|---------------------------|---------|-------------------------------------------------------|---------------------------------------------------------|
| <b>Node.js</b>            | 18+     | Runs JavaScript on the server                         | Same language as the frontend — one team, one language  |
| <b>TypeScript</b>         | 5.6     | Static types on the server                            | Same benefits as frontend                               |
| <b>Express</b>            | 4.21    | Handles HTTP requests and responses                   | Simplest, most popular Node web framework               |
| <b>Prisma</b>             | 5.22    | Talks to the database via typed queries               | Turns SQL rows into TypeScript objects automatically    |
| <b>Zod</b>                | 4.6     | Validates every request body against a schema         | Rejects bad input before it hits the database           |
| <b>bcryptjs</b>           | 3       | Hashes passwords (one-way scrambling)                 | Passwords stored as hashes, never plain text            |
| <b>cookie-parser</b>      | 1.4     | Reads and signs session cookies                       | Foundation of our login system                          |
| <b>Helmet</b>             | 8       | Adds security headers to every response               | Sensible defaults against common web attacks            |
| <b>CORS</b>               | 2.8     | Controls which domains can call the API               | Blocks strangers' websites from calling our backend     |
| <b>Morgan</b>             | 1.10    | Logs every HTTP request                               | So we can see what happened in production               |
| <b>express-rate-limit</b> | 8       | Blocks abusers who send too many requests             | DoS protection                                          |
| <b>Multer</b>             | 2.4     | Handles file uploads (photos)                         | Standard Express file middleware                        |
| <b>ExcelJS</b>            | 4.4     | Generates <code>.xlsx</code> files for report exports | Full XLSX support (styles, columns, sheets)             |
| <b>Papaparse</b>          | 5.7     | Reads/writes CSV files                                | Handles CSV edge cases (commas in values, quotes, etc.) |
| <b>dotenv</b>             | 16      | Loads config from a <code>.env</code> file            | So we don't hard-code secrets                           |

### Development-only extras

- **tsx** — runs TypeScript files directly during development
- **@types/\*** — TypeScript type definitions for the JS libraries above

## 1.4 Database

- **PostgreSQL** — the actual database engine. An industry-standard, ACID-compliant relational database.

- **Neon** — a company that runs Postgres for us. Free tier. We don't manage servers.
- **Prisma** (also listed above) — our "translator" between TypeScript and SQL.

**Why Postgres over SQLite?** SQLite is a file on disk, perfect for local development but hosted apps need a proper always-on database. We started with SQLite and migrated to Postgres when deploying. Same Prisma code works for both.

## 1.5 Hosting & deployment

| Tool              | What it does                                                                     |
|-------------------|----------------------------------------------------------------------------------|
| <b>Render.com</b> | Rents us a virtual machine, runs our Node.js process, serves the site over HTTPS |
| <b>Neon</b>       | Rents us a Postgres database, backups included                                   |
| <b>GitHub</b>     | Stores our source code; every push to <code>main</code> triggers a Render deploy |

The whole pipeline:

```
developer laptop → git push → GitHub → Render notices → Render builds → Render deploys
```

Zero manual steps once configured. See `render.yaml` for the exact recipe.

## 1.6 Development tools

Not shipped in production but essential during development:

| Tool                         | What it does                                                 |
|------------------------------|--------------------------------------------------------------|
| <b>Git</b>                   | Version control — track every change, roll back if needed    |
| <b>npm</b>                   | Installs library dependencies from <code>package.json</code> |
| <b>Prisma CLI</b>            | Runs database migrations, generates the typed client         |
| <b>Chrome DevTools</b>       | Debug the frontend live in the browser                       |
| <b>VS Code (recommended)</b> | Code editor with TypeScript support                          |

# PART 2 — REPOSITORY LAYOUT

## 2.1 Top-level folders

```
upl_project/  
├─ backend/           Node.js API + database schema  
├─ frontend/          React SPA  
├─ docs/              Every document you're reading – including this one  
├─ logo.jpeg          UPL brand image (copied into frontend/public for the app)  
├─ render.yaml        Deployment recipe read by Render.com  
├─ README.md          Repo homepage  
├─ .gitignore         Which files git should ignore  
└─ .env.example       Template for local .env
```

## 2.2 Backend folder

```
backend/  
├─ prisma/  
│  ├─ schema.prisma   Data model (14 tables)  
│  ├─ seed.ts         Loads demo data (roles, admin user, sample equipment)  
│  └─ migrations/     SQL migration files (auto-generated)  
├─ src/  
│  ├─ config/env.ts   Loads and validates environment variables  
│  ├─ controllers/    Thin HTTP layer – parse request, call service  
│  ├─ services/       Business logic – the "brain" of each feature  
│  ├─ routes/         URL → Controller wiring  
│  ├─ middleware/     Runs on every request (auth, logging, rate limit)  
│  ├─ lib/            Shared helpers (Prisma client, error classes, etc.)  
│  ├─ types/          TypeScript type declarations  
│  ├─ app.ts          Express app setup (glues everything together)  
│  └─ server.ts       HTTP listener (main entry point)  
├─ package.json       List of dependencies + npm scripts  
├─ tsconfig.json      TypeScript compiler settings  
└─ .env               Local configuration (gitignored – has secrets)
```

## 2.3 Frontend folder

|                       |                                                   |
|-----------------------|---------------------------------------------------|
| frontend/             |                                                   |
| ├─ public/            | Files served as-is (favicon, logo)                |
| ├─ src/               |                                                   |
| │   ├─ main.tsx       | App entry point – mounts React into the HTML page |
| │   ├─ App.tsx        | Route table – URL → Page component                |
| │   ├─ index.css      | Global Tailwind styles + print rules              |
| │   ├─ auth/          | Session state and route guards                    |
| │   ├─ theme/         | Dark/light mode toggle                            |
| │   ├─ layout/        | App shell (top bar + sidebar)                     |
| │   ├─ pages/         | One file per screen (Dashboard, Login, etc.)      |
| │   ├─ components/    | Reusable pieces (Modal, Button, Badge, ...)       |
| │   └─ lib/           | API client + typed wrappers for each backend area |
| ├─ index.html         | HTML skeleton                                     |
| ├─ package.json       | Dependencies + scripts                            |
| ├─ vite.config.ts     | Build settings                                    |
| ├─ tailwind.config.js | UPL brand palette + dark mode config              |
| ├─ postcss.config.js  | CSS processing pipeline                           |
| └─ tsconfig.*.json    | TypeScript settings                               |

## PART 3 — BACKEND FILES (DEEP DIVE)

### 3.1 The entry point — how a request flows

Before we look at individual files, understand the request pipeline:

1. Browser sends "GET /api/equipment"
2. → server.ts starts the HTTP listener
3. → app.ts passes request through the middleware stack
4. → middleware/\* auth, rate limit, request id, etc.
5. → routes/\* says "this URL goes to controllers/equipmentController.list"
6. → controllers/\* parses query params, calls the service
7. → services/\* runs business logic (permission checks, DB query)
8. → prisma translates to SQL, runs it on Postgres
9. → (response bubbles back up the chain)

Every file below fits somewhere in this pipeline. Understanding the pipeline makes the file list easy to navigate.

## | 3.2 Entry point files

### backend/src/server.ts

**One-liner:** Starts the HTTP listener on a port.

**Why it exists:** Node.js needs an entry file — the one you literally run. This is it. It imports `app.ts` (which builds the Express app) and calls `.listen(port)`.

**Deep dive:** Tiny file. Reads the port from environment variables (defaults to 4000 in dev, or whatever Render assigns in production), prints a "listening on..." message, hooks up graceful shutdown on Ctrl+C. Everything else is delegated to `app.ts`.

**Importance:** Critical. If this crashes, the whole backend is down.

---

### backend/src/app.ts

**One-liner:** Builds the Express application by chaining middleware and mounting routers.

**Why it exists:** Express apps are built by "using" pieces of functionality in a specific order (security first, then auth, then routes, then error handling). This file is where that order is defined.

**Deep dive:** In order, this file: 1. Turns off `X-Powered-By` (security through obscurity — hides which server we run) 2. Sets `trust proxy: 1` (tells Express the app is behind Render's load balancer so `req.ip` returns the real client IP) 3. Applies **Helmet** (adds ~10 security headers) 4. Applies **CORS** (only lets our own frontend call the API) 5. Parses request bodies as JSON 6. Parses signed cookies 7. Assigns a random request ID to every request (for tracing errors) 8. Logs every request with Morgan 9. Runs `attachUser` middleware (loads the current logged-in user) 10. Applies rate limiting 11. Mounts every `routes/*` router at its URL prefix 12. Optionally serves the built frontend static files ( `FRONTEND_DIST_DIR` ) 13. Registers the 404 handler and the global error handler

**Importance:** Critical. Every request touches this file.

---

## | 3.3 Configuration

### backend/src/config/env.ts

**One-liner:** Reads environment variables and gives them typed names elsewhere in the code.

**Why it exists:** You don't want random `process.env.PORT` sprinkled throughout the code. This file is the *single* place where env vars are read, validated, and defaulted.

**Deep dive:** Exports an `env` object with fields like `env.port`, `env.databaseUrl`, `env.sessionSecret`, etc. Uses helper functions `required()` (throws if missing, unless a fallback is provided), `integer()` (parses to number), and `optional()` (returns undefined if missing). Also exports `isProduction` (a boolean derived from `NODE_ENV`).

**Importance:** Critical. Wrong config here → app fails to start or runs with silent bugs (like an empty session secret).

---

## | 3.4 Middleware — code that runs on every request

Middleware sits between the HTTP request arriving and the actual business logic running. Think of it as an assembly line where each station adds or checks something before passing the package on.

`backend/src/middleware/auth.ts`

**One-liner:** Reads the session cookie, looks up the user, attaches them to `req.user`.

**Why it exists:** Almost every backend endpoint needs to know "who is calling?". Doing that lookup in every controller would be repetitive.

**Deep dive:** Contains three exports: - `attachUser` — runs on every request, decodes the signed cookie, looks up the `Session` row, extends the session's `expiresAt`, loads the associated `User`, and puts them on `req.user`. Doesn't fail if there's no session — just sets `req.user = null`. - `requireAuth` — a stricter middleware that returns 401 if `req.user` is null. Applied to routes that need a logged-in user. - `requirePermissions(... perms)` — factory that returns middleware which checks that the user has *all* of the listed permissions.

Also exports `asyncHandler` — a utility that catches async errors from controllers and forwards them to the global error handler (Express 4 doesn't do this natively).

**Importance:** Critical for security. Every protected endpoint uses `requireAuth` + `requirePermissions`.

---

`backend/src/middleware/errorHandler.ts`

**One-liner:** Catches every unhandled error and returns a clean JSON error to the client.

**Why it exists:** Without this, an unexpected error would leak a full stack trace to the browser. This file wraps everything in a structured `{error: {code, message, requestId}}` shape.

**Deep dive:** Two exports: - `notFoundHandler` — catches URLs that no route matched, returns `404 NOT_FOUND`. - `errorHandler` — the last middleware. Inspects the thrown error, returns the right HTTP status: `400` for validation, `401` for auth, `403` for permissions, `404` for not-found, `409` for conflicts, `429` for rate limits, `500` for everything else. Zod errors are turned into human-readable field-level messages. Prisma errors are translated too (`P2002` = unique constraint → 409). In production we hide stack traces.

**Importance:** Critical for both security and user experience.

---

`backend/src/middleware/rateLimits.ts`

**One-liner:** Blocks IP addresses sending too many requests.

**Why it exists:** Without this, a single client (or a bot) could overwhelm the API. Also prevents brute-force login attempts.

**Deep dive:** Exports two limiters: - `globalApiLimiter` — max 300 requests per minute per IP, applied to all `/api/*` . - `writeApiLimiter` — max 60 writes per minute per IP (POST / PUT / DELETE only), applied on top of the global one.

Both are read from env vars ( `RATE_LIMIT_GLOBAL_PER_MINUTE` etc.) so they're tunable without a code change.

**Importance:** High. Prevents accidental and malicious abuse.

---

#### `backend/src/middleware/requestId.ts`

**One-liner:** Assigns a random ID to every request so we can trace it in logs.

**Why it exists:** When an error happens, the client sees a `requestId` in the error response. Ops can grep the server logs for that ID to see exactly what happened for that specific request.

**Deep dive:** Sets `req.id = crypto.randomUUID().slice(0, 12)` . Adds the ID to a response header ( `X-Request-Id` ) and to the Morgan log format via a custom token.

**Importance:** Medium. Not required for the app to work, but essential for debugging in production.

---

#### `backend/src/middleware/upload.ts`

**One-liner:** Configures Multer to handle uploaded image files.

**Why it exists:** File uploads need special HTTP parsing (they use `multipart/form-data` ). Multer does that but needs configuration around max size, allowed MIME types, and storage location.

**Deep dive:** Exports `inspectionPhotoUpload` — Multer middleware that expects a single field named `file` , caps size at `UPLOAD_MAX_BYTES` (default 10 MB), and stores in memory (we then write to disk via `lib/fileStorage.ts` after auth checks pass).

**Importance:** Only needed for inspection photo uploads.

---

## | 3.5 Routes — URL → controller wiring

Every file in `backend/src/routes/` follows the same pattern: create an `express.Router()` , list `router.get('/path', middleware, handler)` lines, export it. `app.ts` mounts each router at a URL prefix.

Most routes are trivial (one to twenty lines each) and share the same shape. Rather than describe each individually, here's the pattern once and a list of what each router covers:

```
export const someRouter = Router();
someRouter.use(requireAuth);
someRouter.get('/', requirePermissions(PERMS.X_VIEW), controller.list);
someRouter.post('/', requirePermissions(PERMS.X_MANAGE), controller.create);
someRouter.get('/:id', requirePermissions(PERMS.X_VIEW), controller.get);
someRouter.put('/:id', requirePermissions(PERMS.X_MANAGE), controller.update);
```

## File list

| File                                      | URL prefix                                                                   | Covers                                                 |
|-------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------|
| <code>routes/health.ts</code>             | <code>/api</code>                                                            | <code>/health</code> — is the DB reachable?            |
| <code>routes/auth.ts</code>               | <code>/api/auth</code>                                                       | Login, logout, <code>me</code> (whoami)                |
| <code>routes/users.ts</code>              | <code>/api/users</code>                                                      | User CRUD + role master (roles, permissions endpoints) |
| <code>routes/units.ts</code>              | <code>/api/units</code>                                                      | Unit CRUD                                              |
| <code>routes/departments.ts</code>        | <code>/api/departments</code>                                                | Department CRUD                                        |
| <code>routes/equipmentTypes.ts</code>     | <code>/api/equipment-types</code>                                            | Equipment type CRUD                                    |
| <code>routes/equipment.ts</code>          | <code>/api/equipment</code>                                                  | Equipment CRUD, QR lookup, bulk import                 |
| <code>routes/checklistTemplates.ts</code> | <code>/api/checklist-templates</code> + <code>/api/checklist-versions</code> | Template + version CRUD, publish                       |
| <code>routes/inspections.ts</code>        | <code>/api/inspections</code>                                                | Start, save, submit, approve, reject, schedule, photos |
| <code>routes/correctiveActions.ts</code>  | <code>/api/corrective-actions</code>                                         | CA CRUD, assign, resolve, close, photos                |
| <code>routes/reports.ts</code>            | <code>/api/reports</code>                                                    | All 7 reports                                          |
| <code>routes/dashboard.ts</code>          | <code>/api/dashboard</code>                                                  | Compliance summary + chart data                        |
| <code>routes/auditLogs.ts</code>          | <code>/api/audit-logs</code>                                                 | Read the audit trail                                   |

**Importance:** Each file is trivial in isolation but essential — without the router, the endpoint isn't reachable.

## | 3.6 Controllers — the "receptionist" layer

A controller does three jobs: 1. **Read** the request (URL params, query string, body) 2. **Validate** the body (using Zod) 3. **Call** the service function 4. **Return** the result as JSON

Controllers never touch the database directly. That's the service's job.

### **File list**

Each controller matches its route file (same resource). Rather than one-by-one, here's what each does at a glance:

| File                                                    | Handlers                                                                                                                          | Notes                                                                                                          |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>controllers/authController.ts</code>              | login, logout, me                                                                                                                 | Sets and clears the session cookie                                                                             |
| <code>controllers/userController.ts</code>              | listUsers, getUser, createUser, updateUser, resetPassword                                                                         | Old <code>listRoles</code> handler still exported but no longer routed (moved to <code>roleController</code> ) |
| <code>controllers/roleController.ts</code>              | listRoles, getRole, listPermissions, createRole, updateRole, deleteRole                                                           | The Role Master API                                                                                            |
| <code>controllers/unitController.ts</code>              | list, get, create, update                                                                                                         | Standard CRUD                                                                                                  |
| <code>controllers/departmentController.ts</code>        | list, get, create, update                                                                                                         | Standard CRUD                                                                                                  |
| <code>controllers/equipmentTypeController.ts</code>     | list, get, create, update                                                                                                         | Standard CRUD                                                                                                  |
| <code>controllers/equipmentController.ts</code>         | list, get, create, update, byQr, applicableChecklist, listInspectionsForEquipment, currentInspectionForEquipment                  | The <code>byQr</code> handler is what QR scans hit                                                             |
| <code>controllers/equipmentImportController.ts</code>   | template download, preview, import                                                                                                | Bulk upload from CSV                                                                                           |
| <code>controllers/checklistTemplateController.ts</code> | Template CRUD, version create, version save, version publish                                                                      | Handles the versioned template model                                                                           |
| <code>controllers/inspectionController.ts</code>        | list, get, listSchedule, start, saveProgress, submit, approve, reject, listPendingApprovals, uploadAttachment, downloadAttachment | The busiest controller — 12+ handlers                                                                          |
| <code>controllers/correctiveActionController.ts</code>  | list, get, create, assign, progress, resolve, close, uploadAttachment                                                             | Corrective-action lifecycle                                                                                    |
| <code>controllers/reportsController.ts</code>           | 7 report handlers, each with JSON / CSV / XLSX output                                                                             | Uses <code>lib/exporters.ts</code> for CSV/Excel                                                               |
| <code>controllers/dashboardController.ts</code>         | summary + chart data                                                                                                              | Just calls the dashboard service                                                                               |
| <code>controllers/auditLogController.ts</code>          | list                                                                                                                              | Simple paginated list                                                                                          |

## Deep dive on the most interesting ones

**controllers/authController.ts** — Login is more than "check password": - Verify username + password against the DB (via **authService**) - Create a **Session** row - Set a signed HTTP-only cookie carrying the session ID - Return the current user + their permissions The cookie is **httpOnly** (JavaScript in the browser can't read it), **secure** (HTTPS only) in production, **sameSite: lax** (protects against cross-site request forgery), and signed (any tampering is detected).

**controllers/inspectionController.ts** — Contains one of the most important flows in the app: **submit**. It calls **service.submitInspection** which either completes the inspection immediately OR sends it to approval, depending on whether the template has approvers.

**controllers/reportsController.ts** — Each report has a **ColumnSpec[]** array defining the CSV/Excel columns. The same array drives both the download format and (for JSON responses) the shape the frontend renders. Adding a column is a one-line change.

**Importance:** Every controller is business-critical for its area.

## 3.7 Services — the "brain" layer

Services own the business logic and are the only layer that talks to the database (via Prisma). They: 1. Enforce **permission** rules (unit scoping, ownership, role checks) 2. Do the actual **database work** 3. Return plain objects the controller can **res.json()**

Every write happens in a **Prisma transaction** ( **prisma.\$transaction** ) so partially-completed changes never survive an error.

### File list & summaries

**services/authService.ts** — Password verification ( **bcrypt.compare** ), session creation, session destruction. Session cookies here are stored server-side too (in the **Session** table), so revoking a session is a real DB row deletion, not just a "log out on the client" wave.

**services/userService.ts** — User CRUD, unit-scope checks (Unit Admin can only touch users in their own unit), password reset (writes a new bcrypt hash). Enforces "only one Super Admin" is *not* enforced — that's a policy call left to operators.

**services/roleService.ts** — Role Master CRUD, plus permission-set replacement. Includes hard guardrails: - **SUPER\_ADMIN** is fully immutable (attempting to change it throws 403). Prevents accidental lock-out. - System roles keep their name and key. - Custom roles can be deleted only when no users are assigned.

**services/unitService.ts** / **services/departmentService.ts** / **services/equipmentTypeService.ts** — Simple CRUD services. Nothing surprising.

**services/equipmentService.ts** — Equipment CRUD with unit-scope enforcement, plus: - **QR value generation** — every new equipment gets a unique random short string (e.g. **EQ-A1B2C3D4E5F6**). - **Equipment code auto-generation** — via **lib/equipmentCodes.ts**, produces sequential codes like **FE-A-0001** scoped per equipment type per unit. - **byQr()** — looks up equipment by its QR value. The heart of the scan-to-inspect flow.

`services/equipmentImportService.ts` — CSV bulk import. Reads the file, validates each row (existence checks for unit/department/type, dedup checks), returns a preview, then commits in one transaction.

`services/checklistTemplateService.ts` — Template + versioning + approvers. The versioning logic is the trickiest part of the whole codebase: - Editing a published template means creating a *new draft version* (cloned from the current published one) - Publishing a draft archives the previous published one and marks the new one as `isCurrent` - Historical inspections reference the exact version they were captured against — template edits never mutate history - Also: approver assignment with role validation ( `Unit Admin` or above)

`services/inspectionService.ts` — The biggest and most complex service in the codebase. Contains: - `startInspection` — creates or resumes a `PENDING` inspection - `saveInspectionProgress` — upserts responses without validation - `submitInspection` — validates all mandatory answers, checks if the template has approvers, either completes the inspection or sends it to `PENDING_APPROVAL` - `completeInspection` — the finalisation step: assigns `INS-YYYY-NNNNNN` number, computes overall result, auto-creates corrective actions for failed safety-critical answers - `approveInspection` / `rejectInspection` — approval workflow - `listSchedule` — the monthly schedule with due/overdue/completed status per equipment - `listPendingApprovals` — for the notification centre

Also contains helpers like `nextInspectionNumber` (per-year monotonic sequence) and `isAnswered` (question-type-aware).

`services/correctiveActionService.ts` — CA lifecycle (OPEN → IN\_PROGRESS → RESOLVED → CLOSED). Includes: - `autoCreateForFailedResponse` — called by `inspectionService` when an inspection completes with a fail on a safety-critical question - Human-readable code generation ( `CA-A-00001` per unit) - Attachment uploads keyed by lifecycle stage (evidence/resolution/closure)

`services/reportsService.ts` — Seven distinct report queries. Each function: 1. Applies the current user's unit-scope 2. Runs the Prisma query 3. Transforms rows into a consistent `{summary, rows, questions?}` shape

The trickiest is `equipmentInspectionLogReport` — it collects every question ever answered in the range, sorts them by section×sequence, and produces one column per question. `equipmentTypeInspectionLogReport` does the same across every equipment of a type.

`services/dashboardService.ts` — Aggregates data for the home dashboard: total equipment, this month's completion %, top failing types, recent activity feed.

`services/auditLogService.ts` — Simple paginated read over the `AuditLog` table with filters.

**Importance:** Services are where business logic lives. Changes here directly affect app behaviour.

## 3.8 Shared helpers ( `lib/` )

Small files with focused purposes. Reused across services.

`backend/src/lib/prisma.ts`

**One-liner:** Exports a single Prisma client instance.

**Why it exists:** Creating a new Prisma client per request would exhaust the DB connection pool. We create one shared instance at app startup.

**Deep dive:** Also handles graceful shutdown ( `await prisma.$disconnect()` on SIGTERM). In dev, hot-reloads reuse the same instance so we don't leak connections between reloads.

---

#### `backend/src/lib/errors.ts`

**One-liner:** Custom error classes with HTTP status codes attached.

**Why it exists:** Services throw semantic errors ( `notFound()` , `forbidden()` , `conflict()` ) that the global error handler can turn into the right HTTP status.

**Deep dive:** Exports `AppError` (base class) + helper factory functions: `badRequest(msg, code)` , `unauthorized` , `forbidden` , `notFound` , `conflict` , `tooLarge` , `rateLimited` . Each carries a symbolic error `code` (e.g. `INVALID_INPUT` ) so the frontend can key UX decisions off the code, not the human-readable message.

---

#### `backend/src/lib/rbac.ts`

**One-liner:** Single source of truth for role and permission keys.

**Why it exists:** Both the seed script and the middleware need to know "what are the 18 permissions and what does each role have?" This file is that source.

**Deep dive:** Exports: - `ROLE_KEYS` — 5 role keys as constants - `PERMISSION_KEYS` — 18 permission keys as constants - `ROLE_PERMISSIONS` — mapping of role → which permissions it gets by default. Used only on first-run seed; operators can change roles later via the Roles UI without those changes being wiped by seed.

**Importance:** Editing this file requires re-seeding on first deploy but is otherwise safe.

---

#### `backend/src/lib/audit.ts`

**One-liner:** Writes rows to the `AuditLog` table.

**Why it exists:** Every meaningful action (login, edit user, submit inspection, close CA, ...) needs to be recorded so we can prove after the fact who did what.

**Deep dive:** Exports: - `AUDIT_ACTIONS` — enum of all action keys - `writeAudit({actorId, action, entityType, entityId, metadata})` — a fire-and-forget function called from controllers after each write

Failure to write an audit row doesn't fail the underlying operation (logged as a warning). That's a deliberate trade-off — you don't want users seeing an "audit log full" error and being unable to work.

---

### backend/src/lib/session.ts

**One-liner:** Cookie signing + session TTL logic.

**Why it exists:** Sessions have sliding expiry (each request bumps the expiry) and an absolute cap (force re-login every 30 days). The math and cookie helpers live here.

**Deep dive:** Exports helpers for building and clearing the session cookie with the right flags ( `httpOnly` , `secure` , `sameSite` , `signed` ). Also computes the next `expiresAt` based on `SESSION_MAX_AGE_MS` .

---

### backend/src/lib/passwords.ts

**One-liner:** Password strength validation + bcrypt hashing helpers.

**Why it exists:** Repeated code (min length, require digit, etc.) consolidated in one place.

**Deep dive:** Exports `hashPassword` (bcrypt with cost 10) and `assertPasswordAcceptable` (throws `INVALID_PASSWORD` if too weak). Called from user creation and password reset flows.

---

### backend/src/lib/fileStorage.ts

**One-liner:** Where uploaded photos go on disk.

**Why it exists:** Multer gives us the file in memory. We need to decide where on disk to write it and what the filename should be.

**Deep dive:** For each inspection, files land in `UPLOAD_DIR/inspections/<inspectionId>/<safe-random-name.jpg>` . Same for corrective-action photos under `.../corrective-actions/...` . Also provides `assertImageAcceptable` — enforces MIME type is `image/*` and size is under `UPLOAD_MAX_BYTES` .

**Caveat:** On Render's free tier, disk is ephemeral — photos disappear on restart. For a real rollout, this file is what you'd rewrite to push to S3 or Cloudflare R2 instead.

---

### backend/src/lib/exporters.ts

**One-liner:** Generates CSV and XLSX files for report exports.

**Why it exists:** Every report needs CSV + Excel output. Rather than duplicating that in each controller, we have a `ColumnSpec[]` shape and `sendCsv` / `sendXlsx` helpers.

**Deep dive:** `ColumnSpec` = `{header: string, accessor: (row) => string|number}` . `sendCsv` uses Papaparse to build the CSV body and streams it out. `sendXlsx` uses ExcelJS to build an actual `.xlsx` binary. Both set the correct `Content-Type` and `Content-Disposition` headers so the browser prompts "Save as..." with a sensible filename.

---

#### `backend/src/lib/inspectionPeriod.ts`

**One-liner:** Turns "October 2026" into a date range (start / end / periodKey / due date).

**Why it exists:** Inspections are bucketed by month (period). We need consistent maths for "what month is now?", "when is the due date for this month?", and "what months fall in this date range?".

**Deep dive:** Key exports: - `currentPeriod()` — returns `{periodKey: '2026-10', periodStart, periodEnd}` for right now - `periodFromKey('2026-10')` — inverse - `computeDueDate(period, frequencyDays)` — end of period for monthly - `resolveDateRange({fromDate, toDate})` — converts a range to a list of month keys, used by reports

---

#### `backend/src/lib/checklistTypes.ts`

**One-liner:** The `QUESTION_TYPES` enum + related helpers.

**Why it exists:** Both the template editor and the inspection form need to know the list of question types. Type guards live here too.

**Deep dive:** Exports `QUESTION_TYPES = ['PASS_FAIL', 'YES_NO', 'NUMERIC', 'TEXT', 'DROPDOWN', 'RADIO', 'CHECKBOX', 'DATE', 'PHOTO', 'REMARKS']`, `isQuestionType()`, `FAILABLE_QUESTION_TYPES`, and `OPTION_BASED_QUESTION_TYPES`.

---

#### `backend/src/lib/correctiveActionTypes.ts` / `backend/src/lib/equipmentCodes.ts`

Small helper files. First defines CA status / priority enums; second generates human-readable equipment codes like `FE-A-0001`.

---

## | 3.9 The database schema

#### `backend/prisma/schema.prisma`

**One-liner:** The definition of every database table, column, and relationship.

**Why it exists:** Prisma reads this file, generates SQL to apply the schema (via `db push` or `migrate`), and generates TypeScript types for the queries.

**Deep dive — 15 tables:**

| Table                      | Purpose                                          |
|----------------------------|--------------------------------------------------|
| Role                       | 5 seeded roles + any custom ones                 |
| Permission                 | 18 fine-grained permissions                      |
| RolePermission             | Join: which permissions each role grants         |
| User                       | Everyone who can log in                          |
| Session                    | One row per active login                         |
| Unit                       | UPL plant locations                              |
| Department                 | Sub-areas within a unit                          |
| EquipmentType              | Fire Extinguisher, Hydrant, etc.                 |
| Equipment                  | Individual pieces of equipment with QR values    |
| ChecklistTemplate          | Logical checklist (name, header, approvers, ...) |
| ChecklistTemplateUnit      | Which units a template applies to                |
| ChecklistTemplateApprover  | Users authorised to approve inspections          |
| ChecklistTemplateVersion   | Immutable-once-published version of a template   |
| ChecklistSection           | A grouping of questions inside a version         |
| ChecklistQuestion          | Individual question with type + options          |
| Inspection                 | One monthly inspection of one equipment          |
| InspectionResponse         | One answer to one question in one inspection     |
| InspectionAttachment       | Photo uploaded during an inspection              |
| CorrectiveAction           | Follow-up task raised on a fail                  |
| CorrectiveActionAttachment | Photo evidence for a CA                          |
| AuditLog                   | Who did what and when                            |

### Design decisions worth knowing:

- **Versioned checklists** — history never mutates. Editing a template creates a new version; old inspections keep the version they were captured against.
- **Snapshotted responses** — each `InspectionResponse` copies the question text and metadata at submission time. If someone deletes a question later, historical reports still make sense.
- **Unique index on (equipmentId, periodKey)** — the DB itself prevents two inspections for the same equipment in the same month.

- `Inspection.inspectionNumber` **unique** — the human-readable `INS-2026-000042` is enforced unique at the DB level.

**Importance:** The heart of the data model. Every service ultimately queries these tables.

---

`backend/prisma/seed.ts`

**One-liner:** Populates the database with initial roles, permissions, users, and demo equipment.

**Why it exists:** A brand-new database is empty. We need at least an admin user to log in and a role/permission set to check against.

**Deep dive:** Runs 8 steps in order: 1. Upsert 18 permissions 2. Upsert 5 roles + assign default permissions 3. Upsert 1 unit + 3 departments (Dahej plant) 4. Upsert 5 demo users (one per role) 5. Upsert 5 equipment types 6. Upsert 10 sample equipment (2 per type) 7. Upsert 5 sample checklist templates 8. Print a summary of counts

The seed is idempotent — safe to run repeatedly. Password hashes are NOT overwritten on re-seed (so operators don't get their passwords reset by a deploy).

**Non-destructive permission update:** For SUPER\_ADMIN, the seed always ensures every permission is granted (prevents lock-out). For other roles, it only seeds permissions on first-time creation. If an operator later edits `INSPECTOR`'s permissions via the Roles UI, a future deploy won't overwrite those edits.

**Importance:** Only runs on first deploy or when re-seeding.

---

## | 3.10 Other backend files

`backend/src/types/express.d.ts`

**One-liner:** Extends Express's `Request` type with our custom fields.

**Why it exists:** We add `req.user`, `req.id`, and (via Multer) `req.file` to Request. TypeScript needs to know about them.

**Deep dive:** A single-purpose type-declaration file. No runtime code. When TypeScript sees `req.user` in a controller, this file tells it what type `user` is.

---

# PART 4 — FRONTEND FILES (DEEP DIVE)

---

## | 4.1 Entry points

`frontend/src/main.tsx`

**One-liner:** Mounts React into the HTML page.

**Why it exists:** Every React app has one file that calls `createRoot( ... ).render(<App />)`. This is it.

**Deep dive:** Also sets up the top-level providers: `ErrorBoundary` (catches React render errors), `ThemeProvider` (dark/light mode), `BrowserRouter` (React Router). Wraps in `React.StrictMode` for extra dev warnings.

**Importance:** Tiny but critical. If this crashes, nothing renders.

---

`frontend/src/App.tsx`

**One-liner:** The URL → Page component route table.

**Why it exists:** React Router needs to know which component to render for each URL. This file is the whole map.

**Deep dive:** Organised in a hierarchy: - Public routes: `/login`, `/forbidden` - Protected routes (require login): everything else - Nested under `<AppLayout />` for the top-bar + sidebar chrome - Each route wrapped in `<ProtectedRoute requirePermissions={ [ ... ] } />` so unauthorized visits redirect to `/forbidden` - 404 catch-all at the end

Also does **code-splitting** on `ScanPage` and `QrBulkPage` — those pull in html5-qrcode (~350 KB) which we don't want in the main bundle.

**Importance:** Critical — the routing hierarchy defines the whole app.

---

`frontend/src/index.css`

**One-liner:** Global CSS: Tailwind imports + print styles.

**Why it exists:** Tailwind needs its `@tailwind base;` etc. directives somewhere. Print stylesheet for the PDF export also lives here.

**Deep dive:** Three sections: 1. `@tailwind base/components/utilities` — pulls in all Tailwind 2. `@layer base` — sets the default font family and body colors, with `dark:` variants so dark mode applies to the whole page 3. `@media print` — hides `.print-hide` elements, keeps `.print-avoid-break` sections whole, adjusts A4 page margins

**Importance:** Foundational. Missing = the app looks unstyled.

---

## | 4.2 Authentication

`frontend/src/auth/AuthContext.tsx`

**One-liner:** Holds the current logged-in user + provides `login()` / `logout()` / `hasPermission()`.

**Why it exists:** Every page needs to know "who am I?" and "am I allowed to see this?". React Context lets the same values flow to every component.

**Deep dive:** On first mount, calls `GET /api/auth/me`. If the cookie is valid, gets back the user. Exposes: - `user` — the current user or `null` - `login(username, password)` — calls the API, updates state - `logout()` — calls the API, clears state - `hasPermission(perm)` — returns true/false, used to hide buttons

**Importance:** Every protected page relies on this.

---

`frontend/src/auth/ProtectedRoute.tsx`

**One-liner:** Redirects unauthenticated / unauthorized users away.

**Why it exists:** Cheap frontend enforcement so users can't even see pages they don't have access to. (Backend still enforces on every API call — this is UX, not security.)

**Deep dive:** A `<Route element={}<ProtectedRoute ... /></Route>` wrapper. If no user → redirect to `/login`. If `requirePermissions` are set and the user doesn't have them → redirect to `/forbidden`. Otherwise render the `<Outlet />` (nested routes).

---

## | 4.3 Theme

`frontend/src/theme/ThemeContext.tsx`

**One-liner:** Dark / light mode toggle with localStorage persistence.

**Why it exists:** Inspectors work in low-light warehouses. Dark mode is a genuine usability feature.

**Deep dive:** On mount, reads the user's saved preference. When the theme changes, adds/removes a `dark` CSS class on `<html>` (that's how Tailwind's `dark:` variants activate) and writes back to localStorage. Also exports a `<ThemeToggle />` button (sun/moon icon) placed in the app's top bar.

**Design note:** Deliberately does *not* follow the system's `prefers-color-scheme` — inspectors on shared devices found the automatic flip confusing.

---

## | 4.4 Layout

`frontend/src/layout/AppLayout.tsx`

**One-liner:** The persistent frame — top bar and left sidebar around every logged-in page.

**Why it exists:** Every authenticated page needs the same chrome. Do it once here, use `<Outlet />` for the varying page content.

**Deep dive:** Renders: - Top bar — logo, SafetyVerse wordmark, mobile menu button, current user info, theme toggle, sign-out button - Sidebar — grouped nav items (Overview, Operations, Masters, Administration, QR Management, Reports, System). Each item is permission-gated — invisible if the user can't access it. On mobile, the sidebar is a drawer that overlays the content. - `print:hidden` — top bar and sidebar disappear when printing so PDF exports are clean.

**Importance:** High. Broken layout = broken app.

---

## | 4.5 Shared components

`frontend/src/components/ui.tsx`

**One-liner:** Reusable UI primitives — Button, Input, Select, Textarea, Badge, PageHeader, EmptyState, ErrorBanner.

**Why it exists:** Every page uses the same styled inputs and buttons. Rather than repeat the Tailwind classes everywhere, centralise them.

**Deep dive:** Each primitive supports `dark:` variants automatically. Button has 4 variants (primary/secondary/danger/ghost). Input, Select, Textarea share a common look with support for labels, hints, and error messages. Badge comes in 5 tones.

**Importance:** Every page imports something from here.

---

`frontend/src/components/Modal.tsx`

**One-liner:** Overlay dialog with fixed header, scrolling body, sticky footer.

**Why it exists:** Every "New X" / "Edit X" form is a modal. Standard pattern.

**Deep dive:** Trapped inside a fixed-position `<div>` that captures clicks on the backdrop to close. `Escape` key also closes. Uses `flex-column` so the body scrolls internally while the footer stays pinned. Handles `body { overflow: hidden }` while open so the page behind doesn't scroll.

---

`frontend/src/components/ErrorBoundary.tsx`

**One-liner:** Catches React rendering errors so the whole app doesn't turn white.

**Why it exists:** A JavaScript error inside a component would blank the entire page by default. Error boundaries let us show a friendly "something went wrong, try refreshing" message instead.

**Deep dive:** Class component (that's how error boundaries work in React 18 — can't use hooks). Wraps the whole app in `main.tsx`. On error, shows a fallback UI with a "Reload" button and (in dev) the error message.

---

`frontend/src/components/DateRangeFilter.tsx`

**One-liner:** Two-date input for report filters (from + to).

**Why it exists:** Every report uses the same date-range widget. DRY.

---

`frontend/src/components/QrCodeCard.tsx` / `QrLabel.tsx`

**One-liner:** SVG-based QR label rendering for print.

**Why it exists:** Equipment labels need to be printable, at the right size, with human-readable info alongside the QR.

**Deep dive:** Uses `qrcode.react` to generate the SVG. `QrLabel` is the full label (QR + equipment code + name + type + location). `QrCodeCard` is a smaller card used in the equipment detail page. Print CSS ensures 1:1 scaling on paper.

---

## | 4.6 API client + typed wrappers

`frontend/src/lib/api.ts`

**One-liner:** The single Axios instance every API call goes through.

**Why it exists:** Consistent base URL, credential handling, error normalisation.

**Deep dive:** Configures Axios with: - `baseUrl` — `/api` in production (backend and frontend share an origin), `http://localhost:4000/api` in dev - `withCredentials: true` — send cookies with every request - `timeout: 15000` — 15-second timeout - Includes a `toApiError()` helper that normalizes any error (network, timeout, backend error) into a `{code, message, status}` shape the UI can render.

**Importance:** Every backend interaction touches this file.

---

`frontend/src/lib/apiXxx.ts` — one per area

Each of these files is thin: TypeScript types matching the backend response shapes, plus wrapper functions that call the right endpoint.

| File                                 | Wraps                                                                        |
|--------------------------------------|------------------------------------------------------------------------------|
| <code>apiChecklists.ts</code>        | Template + version + question endpoints                                      |
| <code>apiCorrectiveActions.ts</code> | CA CRUD + attachments                                                        |
| <code>apiDashboard.ts</code>         | Dashboard summary + chart data                                               |
| <code>apiEquipment.ts</code>         | Equipment CRUD + QR lookup + import                                          |
| <code>apiInspections.ts</code>       | Inspection CRUD + schedule + approve/reject/pending                          |
| <code>apiReports.ts</code>           | 7 report fetch functions + <code>reportExportUrl()</code> for CSV/XLSX links |
| <code>apiUnits.ts</code>             | Unit CRUD                                                                    |
| <code>apiUsers.ts</code>             | User CRUD + role master + permission listing                                 |
| <code>apiAuditLogs.ts</code>         | Audit log paginated list                                                     |

**Deep dive:** Each function looks like:

```
export async function listUnits(): Promise<Unit[]> {
  const { data } = await api.get<{units: Unit[]}>('/units');
  return data.units;
}
```

Boring by design. Any complex logic belongs in the calling page or in the backend service, not here.

**Importance:** The bridge between the frontend and backend. Editing these files is easy and safe.

#### `frontend/src/lib/permissions.ts`

**One-liner:** Mirrors the backend's `RBAC` keys for use in frontend guards.

**Why it exists:** `<ProtectedRoute requirePermissions={[PERMS.USER_VIEW]}>` reads more cleanly than `['user.view']`. Also gives TypeScript autocomplete.

**Deep dive:** Two exports: `ROLES` (5 role keys) and `PERMS` (18 permission keys). Must be kept in sync with `backend/src/lib/rbac.ts` manually — a small copy of duplication that pays for itself in DX.

#### `frontend/src/lib/qr.ts`

**One-liner:** Turns an equipment `qrCodeValue` into a URL for the QR label + parses scanned values back to equipment codes.

**Why it exists:** Both QR generation and scanning need to agree on the URL format.

**Deep dive:** `qrPayload(value)` returns `${window.location.origin}/s/${value}`.

`parseScannedValue(raw)` copes with camera apps that return the whole URL, a bare code, or a URL with query params — extracts the code.

---

## 4.7 Pages — one screen at a time

There's one file per screen. Rather than describe each in isolation, here's the map with a one-liner + notes on the interesting ones.

### Sign-in and error pages

| File                           | What it shows                                             |
|--------------------------------|-----------------------------------------------------------|
| <code>LoginPage.tsx</code>     | Username + password form; UPL logo + SafetyVerse wordmark |
| <code>ForbiddenPage.tsx</code> | "You don't have access to this page"                      |
| <code>NotFoundPage.tsx</code>  | 404 friendly page                                         |

---

**LoginPage deep dive:** Manages `username` / `password` state; on submit calls `login()` from `AuthContext`; redirects to intended page (from `location.state.from`) on success. Shows any error via the normalized `toApiError()`.

---

### Home

`DashboardPage.tsx` — Landing page after login. Shows: - Big stat tiles: total equipment, this month's completion %, overdue count, safety-critical fails - A pie chart of inspection statuses (recharts) - A bar chart of top failing equipment types - Recent activity feed

Uses `fetchDashboardSummary()` from `apiDashboard.ts`. All charts respect dark mode via Tailwind's palette CSS variables.

---

## Masters

| File                                         | Purpose                                                                                        |
|----------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>UnitsPage.tsx</code>                   | List + create + edit units                                                                     |
| <code>DepartmentsPage.tsx</code>             | List + create + edit departments (filter by unit)                                              |
| <code>EquipmentTypesPage.tsx</code>          | List + create + edit equipment types                                                           |
| <code>EquipmentListPage.tsx</code>           | Paginated + searchable table of every equipment; filters by type/unit/status                   |
| <code>EquipmentDetailPage.tsx</code>         | Full equipment view with inspection history, assigned checklist, and QR label                  |
| <code>EquipmentFormModal.tsx</code>          | The "New / Edit equipment" modal shared by list and detail pages                               |
| <code>EquipmentImportPage.tsx</code>         | CSV bulk import — download template, upload file, preview, confirm                             |
| <code>EquipmentLabelPage.tsx</code>          | Printable single-equipment QR label                                                            |
| <code>ChecklistTemplatesPage.tsx</code>      | List + create checklist templates                                                              |
| <code>ChecklistTemplateDetailPage.tsx</code> | Template settings, version list, approvers picker                                              |
| <code>ChecklistVersionEditorPage.tsx</code>  | The heart of template authoring — sections + questions + type-specific fields, drag-to-reorder |

**ChecklistVersionEditorPage deep dive:** The most complex form in the app. - Local `DraftState` is the authoring model (sections → questions with per-type extras) - User-visible types with type-specific extras: - `DROPDOWN` / `RADIO` — options (newline-separated) + optional default - `CHECKBOX` — options only - `NUMERIC` — min / max / unit - Reorder via up/down buttons - On save, POSTs the whole draft as one payload — the backend deletes and rebuilds the section tree in a transaction (avoiding granular add/remove endpoints) - Publish transitions status to `PUBLISHED` and archives the previous current version

## Operations

| File                                        | Purpose                                                                         |
|---------------------------------------------|---------------------------------------------------------------------------------|
| <code>ScanPage.tsx</code>                   | Camera-based QR scanner (html5-qrcode)                                          |
| <code>QrShortRedirectPage.tsx</code>        | Handles the short <code>/s/:value</code> URL that QRs encode                    |
| <code>InspectionsPage.tsx</code>            | Two tabs: monthly schedule + all-inspections history                            |
| <code>InspectionPerformPage.tsx</code>      | The checklist-filling screen — one question at a time                           |
| <code>InspectionDetailPage.tsx</code>       | Read-only view of a completed inspection + PDF print button + approval workflow |
| <code>ApprovalsPage.tsx</code>              | Notification centre — inspections awaiting your review                          |
| <code>CorrectiveActionsPage.tsx</code>      | List of all CAs with priority, status, filters                                  |
| <code>CorrectiveActionDetailPage.tsx</code> | Full CA view — assign, progress, resolve, close, evidence photos                |
| <code>QrBulkPage.tsx</code>                 | Prints many QR labels at once (paginated on A4)                                 |

**InspectionPerformPage deep dive:** The most-used mobile screen. - Loads the inspection detail once - Local `AnswerMap` state — maps `questionId` → `{valueString?, valueNumeric?, valueDate?, attachments[]}` - Section-by-section navigation (tabs on desktop, buttons on mobile) - Question renderers per type ( `PillGroup` for pass/fail, checkbox list for CHECKBOX, etc.) - Photo upload — snaps directly from the phone camera via `<input type="file" accept="image/*" capture="environment">` - Sticky bottom bar with `Save progress` / `Submit inspection` buttons - Confirmation modal on submit: shows the overall predicted result and asks the inspector to type their name

**InspectionDetailPage deep dive:** Read-only until approval: - Displays every question + answer with pass/fail badges - If status is `PENDING_APPROVAL` and user is a listed approver: shows Approve + Reject buttons - If status is `PENDING` with a `rejectionReason` : shows a red banner so the inspector can see what to fix - Print stylesheet turns the whole thing into a form-shaped PDF via the browser's Save-as-PDF dialog

## Administration

| File                       | Purpose                                                             |
|----------------------------|---------------------------------------------------------------------|
| <code>UsersPage.tsx</code> | User CRUD table + reset password modal                              |
| <code>RolesPage.tsx</code> | Role Master — list roles, edit permission sets, create custom roles |

**RolesPage deep dive:** Uses two modals (create + edit). - Permission grid grouped by resource (User, Role, Unit, ...) - Each permission has a human-readable label ("Manage (create / edit / delete)") + its raw key + description - System roles: name and key locked - `SUPER_ADMIN`: entirely locked (not even permissions editable) - Delete button: disabled if the role has users

## Reports

| File                                                  | Purpose                                                                                                     |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>ReportsLandingPage.tsx</code>                   | Card grid of the 7 available reports                                                                        |
| <code>ReportShell.tsx</code>                          | Shared components: <code>ExportBar</code> (CSV/XLSX/Print buttons), <code>ReportSummary</code> (stat tiles) |
| <code>ComplianceReportPage.tsx</code>                 | Per-equipment compliance for a date range                                                                   |
| <code>UnitComplianceReportPage.tsx</code>             | Roll-up per unit                                                                                            |
| <code>EquipmentHistoryReportPage.tsx</code>           | One equipment's inspection history                                                                          |
| <code>EquipmentInspectionLogReportPage.tsx</code>     | One equipment × every question × every inspection                                                           |
| <code>EquipmentTypeInspectionLogReportPage.tsx</code> | Same but across every equipment of one type                                                                 |
| <code>FailedEquipmentReportPage.tsx</code>            | Currently non-compliant equipment                                                                           |
| <code>CorrectiveActionsReportPage.tsx</code>          | CA list with days-open, priority, status                                                                    |

Every report page follows the same pattern: 1. Load filter dropdowns (types, units) 2. Fetch data when filters change 3. Render a `<ReportSummary>` and a table 4. Include an `<ExportBar>` in the header with CSV / Excel / Print links

## System

`AuditLogsPage.tsx` — Paginated audit log with search + entity-type filter. Read-only. Only Super Admin can access.

# PART 5 — CONFIGURATION FILES

## 5.1 Root files

| File                      | Purpose                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>README.md</code>    | Repo intro + local dev instructions                                                                       |
| <code>.gitignore</code>   | Files git should not track ( <code>node_modules</code> , <code>.env</code> , uploaded photos, SQLite DBs) |
| <code>.env.example</code> | Template showing every required env var (without values)                                                  |
| <code>render.yaml</code>  | Render.com "blueprint" — build command, start command, env vars                                           |
| <code>logo.jpeg</code>    | UPL brand image (also copied into <code>frontend/public/</code> )                                         |

**render.yaml deep dive:** Every push to `main` triggers Render to: 1. `cd backend && npm ci --include=dev` — install backend deps 2. `npx prisma generate` — build the typed DB client 3. `cd ../frontend && npm ci --include=dev && npm run build` — build the SPA 4. `cd ../backend && npm run build` — compile backend TS 5. `npx prisma db push --accept-data-loss --skip-generate` — sync schema 6. `npx tsx prisma/seed.ts` — seed (idempotent) 7. `cd backend && node dist/server.js` — start the process

Environment variables are declared with `sync: false` for secrets (set manually in the dashboard) or literal values for non-secrets.

## 5.2 Backend config files

| File                                     | Purpose                                                                                                                                           |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>backend/package.json</code>        | List of dependencies + npm scripts ( <code>dev</code> , <code>build</code> , <code>start</code> , <code>db:seed</code> , <code>db:studio</code> ) |
| <code>backend/tsconfig.json</code>       | TypeScript compiler settings for the whole backend                                                                                                |
| <code>backend/tsconfig.build.json</code> | Additional settings just for the production build                                                                                                 |
| <code>backend/package-lock.json</code>   | Exact resolved dependency versions (not human-edited)                                                                                             |
| <code>backend/.env</code> (local only)   | Real values for env vars, gitignored                                                                                                              |

## 5.3 Frontend config files

| File                                     | Purpose                                                                                          |
|------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>frontend/package.json</code>       | Frontend dependencies + scripts ( <code>dev</code> , <code>build</code> , <code>preview</code> ) |
| <code>frontend/vite.config.ts</code>     | Vite settings — how the app is built and dev-served                                              |
| <code>frontend/tailwind.config.js</code> | UPL orange brand palette + <code>darkMode: 'class'</code>                                        |
| <code>frontend/postcss.config.js</code>  | PostCSS pipeline (feeds Tailwind + autoprefixer)                                                 |
| <code>frontend/tsconfig.json</code>      | Base TypeScript settings                                                                         |
| <code>frontend/tsconfig.app.json</code>  | Settings for the app source                                                                      |
| <code>frontend/tsconfig.node.json</code> | Settings for the Vite config file (which runs in Node, not the browser)                          |
| <code>frontend/index.html</code>         | HTML skeleton — <code>&lt;div id="root"&gt;</code> where React mounts                            |

**tailwind.config.js deep dive:** Defines the `brand.*` colour scale ( `#F37021` at 500 with lighter/darker shades). Enables `darkMode: 'class'` so we can toggle a `dark` class on `<html>`. Also defines the default sans-serif font stack.

---

## APPENDIX A — File count by area

| Area                           | Files                                        |
|--------------------------------|----------------------------------------------|
| Backend controllers            | 14                                           |
| Backend services               | 14                                           |
| Backend routes                 | 13                                           |
| Backend middleware             | 5                                            |
| Backend lib (helpers)          | 12                                           |
| Backend config + entry         | 3                                            |
| Prisma (schema + seed)         | 2                                            |
| Frontend pages                 | ~30                                          |
| Frontend shared components     | 6                                            |
| Frontend API client wrappers   | 10                                           |
| Frontend auth + theme + layout | 4                                            |
| Frontend entry + routes        | 3                                            |
| Config files (both packages)   | ~10                                          |
| <b>Total</b>                   | <b>~125</b> source files, ~20k lines of code |

## APPENDIX B — Where to look when you want to change X

| I want to change...           | Look here                                                                                       |
|-------------------------------|-------------------------------------------------------------------------------------------------|
| The colour scheme             | <code>frontend/tailwind.config.js</code>                                                        |
| The database schema           | <code>backend/prisma/schema.prisma</code>                                                       |
| A permission                  | <code>backend/src/lib/rbac.ts</code> + <code>frontend/src/lib/permissions.ts</code>             |
| A URL                         | <code>backend/src/routes/*.ts</code>                                                            |
| A rule about who can do what  | <code>backend/src/services/*.ts</code>                                                          |
| Input validation              | <code>backend/src/controllers/*.ts</code> (Zod schemas)                                         |
| What appears on a page        | <code>frontend/src/pages/*.tsx</code>                                                           |
| Reusable UI style             | <code>frontend/src/components/ui.tsx</code>                                                     |
| The list of question types    | <code>backend/src/lib/checklistTypes.ts</code> + <code>frontend/src/lib/apiChecklists.ts</code> |
| Report columns                | <code>backend/src/controllers/reportsController.ts</code> ( <code>ColumnSpec[]</code> )         |
| Deployment settings           | <code>render.yaml</code>                                                                        |
| Environment variable defaults | <code>backend/src/config/env.ts</code>                                                          |

## APPENDIX C — Recommended reading order

If you're new to the codebase, read the files in this order:

1. `README.md` — general orientation
2. `render.yaml` — how it deploys
3. `backend/prisma/schema.prisma` — the data model
4. `backend/src/lib/rbac.ts` — role and permission vocabulary
5. `backend/src/app.ts` — request pipeline
6. `backend/src/services/inspectionService.ts` — the most complex service; if this makes sense, the rest is easy
7. `frontend/src/App.tsx` — route table
8. `frontend/src/lib/api.ts` — how the frontend talks to the backend

9. `frontend/src/pages/InspectionPerformPage.tsx` — the most-used screen, best example of a real feature end-to-end
10. Any one report page + `reportsService.ts` + `reportsController.ts` — best example of the full backend pipeline

By the end, you'll have a working mental model of the whole system.